



VNW Project

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

2 de Abril de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	3
5. Casos de prueba	5
6. Ejemplos	7

1. Equipo

Nombre	Apellido	Legajo	E-mail
Mateo	Andrioli	64042	mandrioli@itba.edu.ar
Tomás	Kaneko	62297	tkaneko@itba.edu.ar
Antonio	Valentinuzzi	64751	avalentinuzzi@itba.edu.ar
Francisco	Peña	62431	fpenacritto@itba.edu.ar

2. Repositorio

La solución será versionada en: https://github.com/matsdah/TPE_TLA.

3. Dominio

El dominio seleccionado para el diseño e implementación de este Lenguaje Específico de Dominio (DSL) comprende la creación, estructuración y orquestación de Novelas Visuales (o *Visual Novels*) y videojuegos con una fuerte relación narrativa.

Una Novela Visual es un género de videojuego interactivo caracterizado por enfocarse casi exclusivamente en contar una historia a través de texto, acompañado de imágenes estáticas o animadas (fondos y *sprites* de personajes), efectos de sonido y música. La interactividad radica principalmente en la toma de decisiones por parte del jugador, lo que genera ramificaciones en la trama o *branching narrative*, afectando variables de estado internas (como la afinidad con un personaje, el karma, el progreso en una ruta específica, economía, etc.) y conduciendo a múltiples finales posibles.

El lenguaje propuesto abstrae las complejidades subyacentes del desarrollo de videojuegos (ciclo principal de juego o *game loop*, renderizado gráfico, gestión de recursos multimedia, manejo de eventos, etc.) y expone un modelo computacional centrado puramente en la narrativa. El lenguaje es de tipado estático con variables del tipo *strong typing*.

El dominio es abstraído a través de entidades de primera clase como: actores, escenas, recursos (o *assets*), flujos de decisión, o estados (variables numéricas que se modifican en base a las decisiones del jugador).

4. Construcciones

El lenguaje provee un conjunto de construcciones diseñadas específicamente para abstraer el ciclo de vida de una novela visual. Para reducir la fricción para los diseñadores, la sintaxis prioriza la legibilidad y un enfoque declarativo.

Antes de manipular el flujo narrativo, el lenguaje permite declarar las entidades fundamentales (actores y recursos) y manejar el estado interno del juego mediante variables gracias a las siguientes construcciones:

- (I). **Declaración de personajes o actores:** Define una entidad capaz de emitir diálogos. Se vincula un nombre de visualización con un identificador interno y metadatos opcionales.
- (II). **Declaración de Recursos:** Vincula un archivo multimedia (imagen, vídeo, pista de audio) a un identificador para su posterior invocación.
- (III). **Asignación y Mutación de Variables:** Declarar o modificar variables de estado (como afinidad, puntos, o economía). Soporta operadores de asignación directa (=) y acumulación (+=, -=).
- (IV). **Bloques de Escena** (*scene*): Actúa como el *scope* principal y el punto de entrada para un fragmento de la historia. Todo el código ejecutable debe residir dentro de una escena.
- (V). **Bifurcación Interactiva** (*choice*): Estructura principal para la toma de decisiones. Pausa la ejecución secuencial y presenta varias opciones al jugador. Cada bloque de opción contiene su propio *scope* de ejecución.
- (VI). **Bifurcación Condicional** (*if, else*): Evalúa el estado de las variables para alterar dinámicamente el flujo de la historia sin la intervención directa del jugador (por ejemplo, desbloquear un diálogo secreto si el carisma es alto).
- (VII). **Salto Incondicional** (*goto*): Transfiere el control del flujo de ejecución hacia el inicio de otra escena definida. Fundamental para conectar las distintas ramas de la historia.
- (VIII). **Salida de texto** (Diálogos): Invoca el identificador de un actor previamente declarado seguido de una cadena de texto o *string*. Le indica al motor que debe renderizar la caja de diálogo.
- (IX). **Salida Gráfica** (*show, hide*): Manipula la visualización de los recursos en pantalla. Permite mostrar u ocultar fondos y *sprites* de personajes.
- (X). **Salida de Audio** (*play, stop*): Controla la reproducción de sonidos, diferenciando conceptualmente entre efectos de sonido (reproducción única) y música de fondo (reproducción en bucle).

- (XI). **Entrada del Jugador** (*Input* implícito): A diferencia de lenguajes de propósito general que requieren rutinas complejas para leer el teclado o ratón, en este el *input* es manejado implícitamente por el motor gráfico durante el bloque *choice* y en la transición entre líneas de diálogo.
- (XII). **Finalización de programa:** En vista de que el programa puede terminar en cualquier momento por decisión del developer, se debe poder dar fin al programa y que no siga corriendo.
- (XIII). **Cargado de archivos de escenas, actores y recursos genéticos:** Para mejorar la implementación del código y evitar archivos con gran cantidad de escenas. El lenguaje debe ser capaz de obtener actores, escenas y recursos genéricos que no se encuentren en el mismo archivo original pero que se encuentren referenciado correctamente su origen.

5. Casos de prueba

Para garantizar la robustez del compilador y la correcta validación del modelo computacional abstraído, se definen los siguientes programas de prueba. El analizador sintáctico (*Flex* y *Bison*) y el analizador semántico deberán discriminar correctamente entre programas bien formados y aquellos que violen las reglas del lenguaje.

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Declaración básica de entidades y ejecución de escena lineal.
- (II). Escena interactiva con múltiples opciones (*branching simple*).
- (III). Mutación y evaluación condicional de una variable de estado entera.
- (IV). Múltiples saltos incondicionales (*goto*) entre diferentes escenas.
- (V). Reproducción y detención de recursos de audio (*play, stop*).
- (VI). Intercalado de visualización y ocultamiento de recursos gráficos (*show, hide*).
- (VII). Evaluación de estructuras condicionales anidadas (*if/else* dentro de *if/else*).
- (VIII). Operaciones aritméticas y acumulación sobre múltiples variables simultáneas.
- (IX). Intercambio de diálogos consecutivos entre múltiples actores declarados.
- (X). Escena con ramificación interactiva cíclica (bucle narrativo cerrado).
- (XI). Acceso a todos los recursos de un archivo a otro.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Invocación de un actor o recurso no declarado en el ámbito global.
- (II). Salto incondicional (*goto*) hacia un identificador de escena inexistente.
- (III). Incompatibilidad de tipos en expresiones matemáticas o de asignación.
- (IV). Presencia de sentencias ejecutables fuera del bloque de una escena (violación de *scope*)

- (V). Uso de palabras reservadas del lenguaje como identificadores de variables o actores.

6. Ejemplos

En esta sección se exponen fragmentos de código representativos de nuestro DSL, diseñados para ilustrar la sintaxis y el comportamiento esperado del modelo computacional abstraído.

I. Declaración de entidades y flujo lineal

Este primer ejemplo demuestra la sintaxis fundamental para declarar recursos en el ámbito global y cómo se orquesta una escena secuencial básica con diálogos y manipulación de recursos multimedia.

```
// Declaración de recursos y actores en el ámbito global
character "Narrador" as narr color "□ #FFFFFF";
character "Aria" as aria color "■ #FF5733";

asset bg_bosque = "assets/img/bosque_oscurο.png";
asset bgm_misterio = "assets/audio/tema_misterio.ogg";

// Definición de una escena lineal
scene "Introduccion_Bosque" {
  show background bg_bosque;
  play music bgm_misterio;

  narr "El viento soplaba fríamente entre los árboles milenarios."
  aria "No deberíamos estar aquí... Siento que nos observan."
  narr "De repente, una rama se quebró a sus espaldas."
}
```

El código inicializa dos actores (*narr* y *aria*) asignándoles un nombre de visualización y un color. Luego, carga una imagen de fondo y una pista de audio. Dentro del bloque *scene*, las instrucciones se ejecutan de manera secuencial; mostrando el fondo, reproduciendo la música y renderizando las líneas de diálogo en el orden especificado.

II. Ramificación interactiva y manejo de estado

Este segundo ejemplo ilustra la capacidad del lenguaje para manejar lógica condicional, toma de decisiones por parte del jugador y saltos entre distintas escenas.

```
// Declaración de variable de estado
set afinidad_narr = 5;

scene "Decision_Cueva" {
    aria "Hay dos caminos posibles. ¿Por cuál deberíamos ir?"

    // Bifurcación interactiva
    choice {
        "Ir por el sendero iluminado" {
            set afinidad_narr += 1;
            aria "Me parece una decisión sensata."
            goto "Escena_SenderoSeguro";
        }
        "Adentrarse en la oscuridad" {
            set afinidad_narr -= 2;
            aria "Estás loco si crees que iré por ahí."

            // Evaluación condicional del estado
            if (afinidad_narr < 5) {
                aria "¡Me voy de aquí! Sobrevive por tu cuenta."
                goto "Escena_Abandono";
            } else {
                aria "Supongo que no me queda otra que seguirte..."
                goto "Escena_CuevaOscura";
            }
        }
    }
}
```

Se define una variable numérica global (*afinidad_narr*). En la escena, la estructura *choice* detiene la ejecución para que el usuario tome una decisión. Dependiendo de la rama elegida, se muta la variable de estado utilizando los operadores *+=*, *-=*.

En la segunda opción, se demuestra el uso de un bloque condicional *if/else* que evalúa el estado interno para realizar un salto incondicional (*goto*) hacia diferentes escenas, alterando dinámicamente el grafo narrativo.